

## AUFGABE ZUR VORLESUNG

# Übung 03 – Eigenes Image und Umgebungsvariablen

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge  
Thema Docker: Dockerfile, Layer, Konfiguration per Environment  
Dozent Dr. Ulrich Winkler  
Semester 5. Semester · Informationstechnik

**Vorlesung:** Deck *docker-basics* **Ziel:** Aus dem `log_tool` ein **eigenes Image** bauen (statt bei jedem Lauf per Bind Mount zu arbeiten) und das Verhalten über **Umgebungsvariablen** konfigurieren.

### Kontext

Bisher haben Sie den Code bei jedem Start hineingemountet. Jetzt verpacken Sie ihn fest in ein **eigenes Image**. Dazu dient das `Dockerfile` in diesem Ordner. Ein Image besteht aus **Layern**: jede `Dockerfile` -Anweisung erzeugt eine Schicht, und Docker verwendet unveränderte Schichten beim nächsten Build wieder (Cache).

Die `.dockerignore` sorgt dafür, dass unnötige Dateien nicht in den Build-Kontext wandern.

### Aufgabenstellung

1. Sehen Sie sich `Dockerfile` und `.dockerignore` an. Bauen Sie daraus ein Image mit dem Namen `log-tool:1.0`.
2. Führen Sie das Image aus (mit einem Volume für die Daten) und legen Sie ein paar Einträge an. Listen Sie sie auf.
3. Ändern Sie über die Umgebungsvariable `LOG_FILE` den Speicherort der Logdatei (z. B. `/data/andere.txt`) und prüfen Sie, dass in eine andere Datei geschrieben wird.
4. Ändern Sie eine Zeile in `log_tool.py` und bauen Sie erneut. Beobachten Sie in der Build-Ausgabe, welche Layer **aus dem Cache** kommen und welche neu gebaut werden.
5. Sehen Sie sich Ihr Image in Portainer (**Images**) und mit `docker image ls` an.

Tipp: Reihenfolge im Dockerfile zählt für den Cache – selten geänderte Dinge nach oben, oft geänderten Code nach unten. Deshalb steht `COPY log_tool.py` möglichst spät.

### Musterlösung

```
cd uebungen/03_image_und_env

# 1) Image bauen
docker build -t log-tool:1.0 .

# 2) Ausführen mit Datenvolume (ENTRYPOINT ist "python log_tool.py",
#     wir hängen nur noch add/list an)
docker run --rm -v logdaten:/data log-tool:1.0 add "Aus eigenem Image"
docker run --rm -v logdaten:/data log-tool:1.0 add "Zweiter Eintrag"
docker run --rm -v logdaten:/data log-tool:1.0 list
```

```
# 3) Speicherort per Environment umstellen
docker run --rm -v logdaten:/data -e LOG_FILE=/data/andere.txt \
  log-tool:1.0 add "Landet woanders"
docker run --rm -v logdaten:/data -e LOG_FILE=/data/andere.txt \
  log-tool:1.0 list

# 4) log_tool.py minimal ändern, dann:
docker build -t log-tool:1.1 .
#   -> "CACHED" bei den oberen Layern (FROM, WORKDIR), NEU ab COPY log_tool.py

# 5) Images ansehen
docker image ls | grep log-tool
```

**Was Sie gelernt haben:** `docker build` erzeugt aus einem `Dockerfile` ein wiederverwendbares Image; Images bestehen aus gecachten Layern; `.dockerignore` hält den Build schlank; `ENV / -e` konfigurieren einen Container, ohne den Code zu ändern.